

Inigo Quilez :: articles :: smooth minimum - 2013



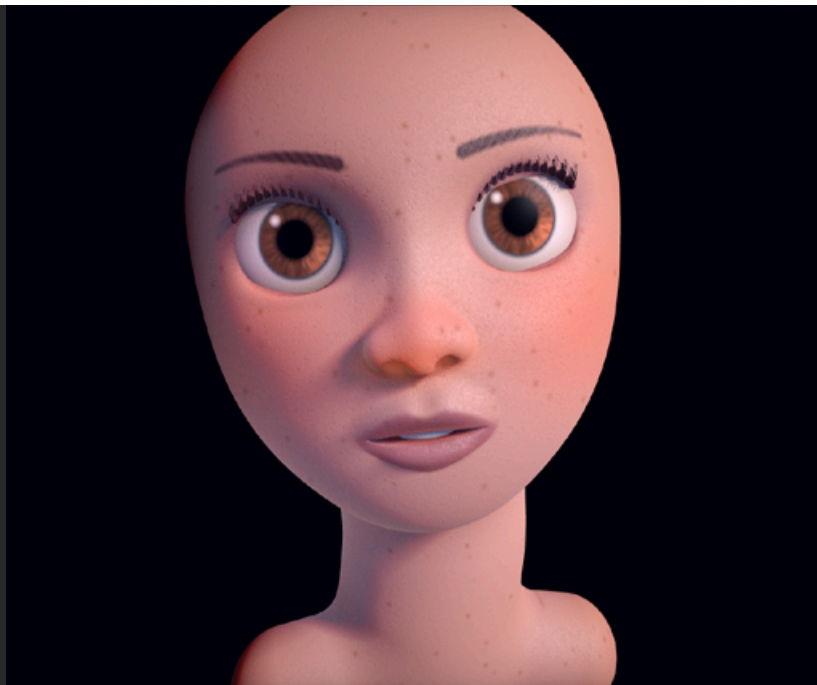
## Intro

---

One of the basic building blocks of Signed Distance Field (SDF) modeling based on **basic primitives** (as opposed to Grids or Neural Networks) is the Smooth-Minimum operator or Smooth-Union. This is similar to a regular Minimum/Union operator, but, well "smooth". This means that where the regular or non-smooth Minimum/Union takes two input SDFs **a** and **b** and returns the closest one of the two, effectively combining both into a single field (for the distance to the combined shape is the distance to whichever piece is closest), the Smooth version combines the two shapes by blending and melting them together, if they are close enough. This helps not only aggregate shapes together, but sculpt organic shapes with them, as if the primitives were made of clay.



Minimum



Smooth-minimum

The images above are an example of using the smooth minimum to sculpt a human face. First we have the regular minimum or union based field, where all the base primitives are clearly visible. Then we can see the Smooth-minimum or Smooth-union, where the shapes have blended together to form one continuous surface. You can see it work in realtime and explore the code in [Shadertoy](#).

## A list of Smooth-minimums

---

So the regular Minimum simply takes the lowest valued of two SDFs **a** and **b**:

```
float min( float a, float b )
{
    return (a<b) ? a : b;
}
```

Most languages come with this function already defined, either as a hardware instruction (most likely) or through a software emulation with an actual branch like in the code above (most unlikely).

Now, the Smooth-minimum must achieve a smooth, non-binary transition between the values of **a** and **b** for regions where **a** and **b** are close enough to each other within some tolerance **k** (measured in regular, distance units). So, when functions or surfaces are close enough to each other, they will blend or melt together.

We can design an infinite number of such functions, each with different pros and cons, but here's a list of some such possible smooth-minimum functions that I've used at one point or another:

```
// exponential
float smin( float a, float b, float k
{
    k *= 1.0;
    float r = exp2(-a/k) + exp2(-b/k);
    return -k*log2(r);
}
```

```
// root
float smin( float a, float b, float k
{
    k *= 2.0;
    float x = b-a;
    return 0.5*( a+b-sqrt(x*x+k*k) );
}
```

```
// sigmoid
float smin( float a, float b, float k
{
    k *= log(2.0);
    float x = b-a;
    return a + x/(1.0-exp2(x/k));
}
```

```
// quadratic polynomial
float smin( float a, float b, float k
{
    k *= 4.0;
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - h*h*k*(1.0/4.0);
}
```

```
// cubic polynomial
float smin( float a, float b, float k
{
    k *= 6.0;
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - h*h*h*k*(1.0/6.0);
}
```

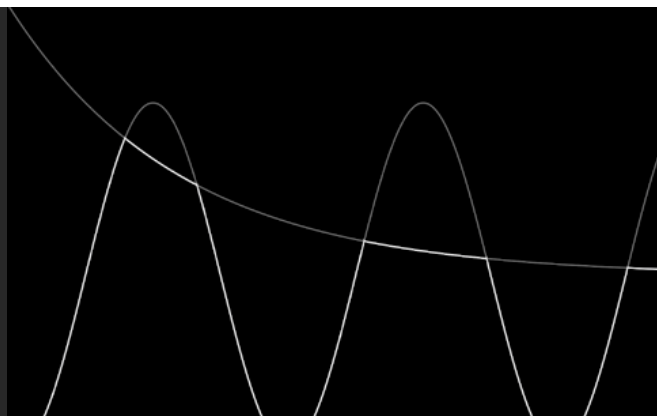
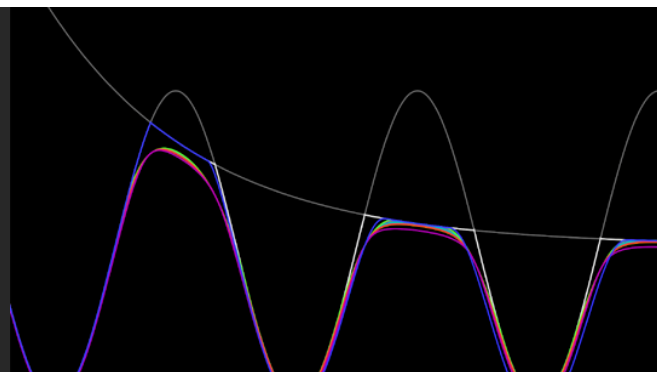
```
// quartic polynomial
float smin( float a, float b, float k
{
    k *= 16.0/3.0;
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - h*h*h*(4.0-h)*k;
}
```

```
// circular
float smin( float a, float b, float k
{
    k *= 1.0/(1.0-sqrt(0.5));
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - k*0.5*(1.0+h-sqr
}
```

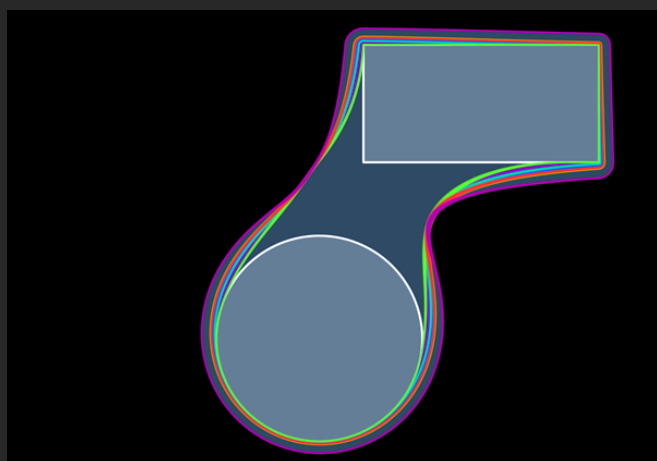
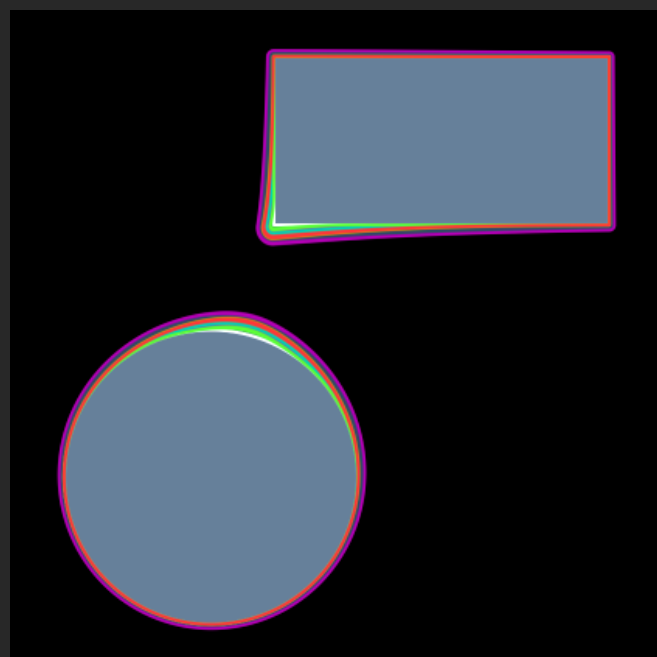
```
// circular geometrical
float smin( float a, float b, float k
{
    k *= 1.0/(1.0-sqrt(0.5));
    return max(k,min(a,b)) -
        length(max(k-vec2(a,b),0.0))
}
```

You have the code for the functions above here [in this Shadertoy example](#).

Now let's just have a look to their behavior first with  $a(x)=e^{-x}$  and  $b(x)=\sin(4x)$  ([quadratic](#), [cubic](#), [quartic](#), [circular](#), [exponential](#), [sigmoid](#), [square root](#), [circular geometrical](#)):

 $\min(a(x), b(x))$  $\text{smin}(a(x), b(x))$ 

When applied to shapes, they look like this:

Moving shapes blended with constant  $k$ Stationary shapes blended with varying  $k$ 

On the left you can see a moving circle and a rectangle blended together by the different smooth-minimum functions defined above with a constant  $k$ , and on the right you see the effect of modifying the parameter  $k$  over two constant shapes. The different colors of the shapes correspond to the different smooth-minimums above.

Something interesting thing to note is, perhaps, that for any given parameter  $k$ , these smooth-minimum functions produce blending regions of approximately the same size. This is thanks to the normalization factors I've introduced for  $k$  in each variant. So let's talk about that next.

## Normalization, thickness and bounds

---

If you've been using smooth-minimum functions before in Shadertoy or elsewhere, you probably have seen some of these smooth-minimum functions above (most likely the Quadratic one) but without the premultiplication of  $k$  that I have introduced here. This is a "normalization" factor that I've computed so that the parameter  $k$  always maps directly to the thickness of the blended area, in actual distance units. This equalizes all smooth-minimums and makes them more or less compatible and interchangeable with each other. You can see this effect in the animations above, where the neck connecting the circle and the rectangle has constant thickness regardless of the smooth-minimum in use.

The computation of this normalization factor is easy - you need to consider the deviation of the smooth-minimum from the minimum when  $a = b$ . Later we'll learn that this is precisely the value of each smooth-minimum's Kernel function, evaluated at the origin. But for now, I hope it makes sense to you that when normalized this way, the value of the parameter  $k$  matches exactly the maximum inflation or thickening that the shapes  $a$  and  $b$  undergo due to the smooth blending happening. Which means that in this normalized form, the parameter  $k$  is also the bounding box expansion required to perfectly bound the smooth-minimum:



Here the circle  $a$  and the rectangle  $b$  have been expanded by  $k$  units to produce a combined bounding volume

in blue that touches exactly the shape of the smooth-minimum. Being able to perform this computation is hence key to getting good hierarchical acceleration structures like BVHs and grids for raycasting, proximity queries and collision detection.

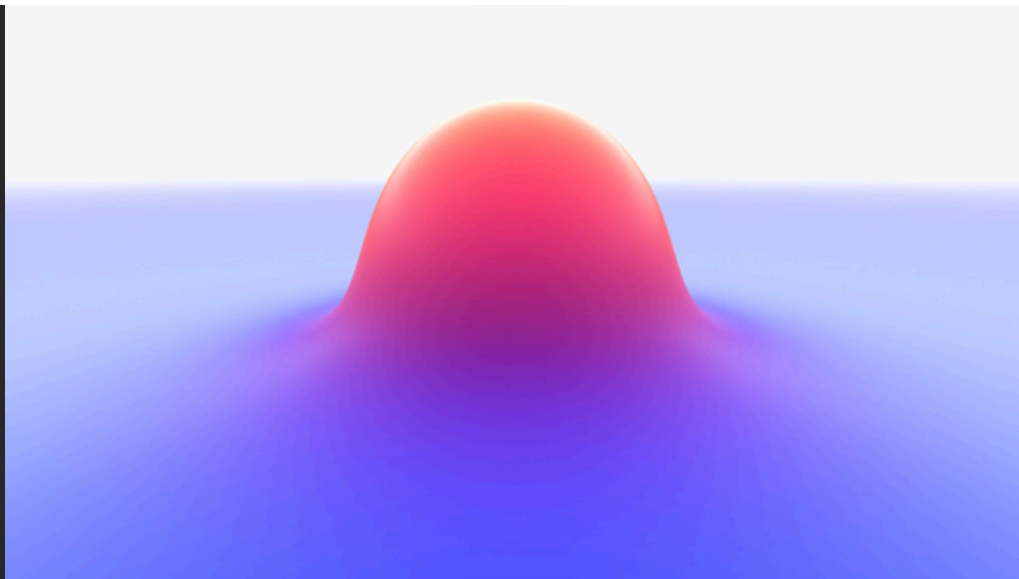
## Mix factor

---

Besides smoothly blending values, it's often also useful to share the blending factor to mix the materials of the two SDFs involved. As an example, in the image below I'm mixing the red and a blue materials based on this blending factor as computed by the code below, which returns the smooth-minimum in *.x* and the blend factor in *.y* for the Quadratic and the Cubic smooth-minimums. Please note I have propagated the **g(0)** normalization already into the expressions, to get some extra simplifications and regularity. You can derive the blend factors for the other smooth-minimum functions rather easily too, I leave it to you as an exercise:

```
// quadratic polynomial
vec2 smin( float a, float b, float k )
{
    float h = 1.0 - min( abs(a-b)/(4.0*k), 1.0 );
    float w = h*h;
    float m = w*0.5;
    float s = w*k;
    return (a<b) ? vec2(a-s,m) : vec2(b-s,1.0-m);
}
```

```
// cubic polynomial
vec2 smin( float a, float b, float k )
{
    float h = 1.0 - min( abs(a-b)/(6.0*k), 1.0 );
    float w = h*h*h;
    float m = w*0.5;
    float s = w*k;
    return (a<b) ? vec2(a-s,m) : vec2(b-s,1.0-m);
}
```



Mixing materials with smooth-minimum

[Code in Shadertoy](#)

## The DD Family

---

Okey, time to look into the smooth-minimum functions more in depth. We'll begin by recognizing that most of the functions I introduced at the beginning of the article actually belong to one family that generalizes them. The code above has been written in more or less optimized GPU code, which obfuscates the similarities between them, but if we rewrite them a bit, we'll see that the **root**, **sigmoid**, **quadratic**, **cubic**, **quartic** and **circular** variants are very similar and belong to the same family:

```
// root
float smin( float a, float b, float k
{
    k *= 2.0;
    float x = (b-a)/k;
    float g = 0.5*(x+sqrt(x*x+1.0));
    return b - k * g;
}
```

```
// sigmoid
float smin( float a, float b, float k
{
    k *= log(2.0);
    float x = (b-a)/k;
    float g = x/(1.0-exp2(-x));
    return b - k * g;
}
```

```
// quadratic polynomial
float smin( float a, float b, float k
{
    k *= 4.0;
    float x = (b-a)/k;
```

```
// cubic polynomial
float smin( float a, float b, float k
{
    k *= 6.0;
    float x = (b-a)/k;
```

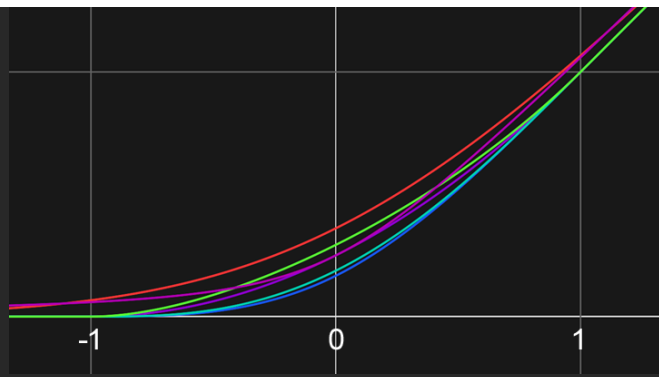
|                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> float g = (x &gt; 1.0) ? x :           (x &lt; -1.0) ? 0.0 :           (x*(2.0+x)+1.0)/4.0; return b - k * g; }  // quartic polynomial float smin( float a, float b, float k {     k *= 16.0/3.0;     float x = (b-a)/k;     float g = (x &gt; 1.0) ? x :               (x &lt; -1.0) ? 0.0 :               (x+1.0)*(x+1.0)*(3.0-x*( return b - k * g; } </pre> | <pre> float g = (x &gt; 1.0) ? x :           (x &lt; -1.0) ? 0.0 :           (1.0+3.0*x*(x+1.0)-abs(x return b - k * g; }  // circular float smin( float a, float b, float k {     k *= 1.0/(1.0-sqrt(0.5));     float x = (b-a)/k;     float g = (x &gt; 1.0) ? x :               (x &lt; -1.0) ? 0.0 :               1.0+0.5*(x-sqrt(2.0-x*x) return b - k * g; } </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

These are mathematically equivalent versions of the ones introduced earlier, you can probably play a bit with pen and paper and rearrange the terms in the formulas to convince yourself that this is true. Indeed they can be written in either normal form (left) or the optimized form (right):

|                                                                                                                       |                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> float smin( float a, float b, float k {     k /= g(0.0);     float x = (b-a)/k;     return b - k*g(x); } </pre> | <pre> float smin( float a, float b, float k {     k /= g(0.0);     float h = max(k-abs(a-b),0.0)/k;     return min(a,b) - k*g(h-1.0); } </pre> |
|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|

First note that all the smooth-minimums in this family are, through the use of `min()` and the kernel, a function of the direct difference of the SDFs **a** and **b**. That's why I called this the Direct Difference Family (or DD Family).

Now, the function **g(x)** is the only thing that's different among all the smooth-minimum of this family. I call this function **g(x)** the **Kernel** of the smooth-minimum. The Kernel plays the important role of defining many of the characteristics of the smooth-minimum, and we'll later in the article design a few interesting such kernels. For now, these are the graphs of a few of the kernels above ([quadratic](#), [cubic](#), [quartic](#), [circular](#), [sigmoid](#), [square root](#)):



A few DD kernels

Kernel:

Normalization factor:

$$g(x) = \frac{(x \cdot (2+x)+1)}{4}$$

$$g(0)=1/4$$

$$g(0)=1/6$$

$$g(x) = \frac{(1+3x \cdot (x+1)-|x^3|)}{6}$$

$$g(0)=3/16$$

$$g(0)=1-\sqrt{1/2}$$

$$g(x) = \frac{(x+1)^2(3-x \cdot (x-2))}{16}$$

$$g(0)=1$$

$$g(0)=1/2$$

$$g(x) = 1 + \frac{1}{2}(x - \sqrt{(2-x^2)})$$

$$g(x) = x/(1-2^{-x})$$

$$g(x) = \frac{(x + \sqrt{(x^2+1)})}{2}$$

The shape of the kernel  $g(x)$  is very specific. When  $x$  tends towards infinity, ie, when  $b$  is much larger than  $a$ ,  $g(x)$  tends towards the identity  $x$  such that the smooth-minimum returns  $a$ . On the other hand, when  $x$  tends towards negative infinity, which happens when  $a$  is much larger than  $b$ , then  $g(x)$  tends towards 0.0 such that the smooth-minimum tends towards  $b$ . Basically, the Kernel behaves like the function  $\max(x, 0)$ , or ReLU if you are a Machine Learning person, but in a relaxed way. That's where the smoothness of the smooth-minimum comes from. In fact, if we make  $g(x) = \max(x, 0)$ , a perfect rectifier, then the smooth-minimum reduces, mathematically, to a regular minimum.

The kernel plays a very important role in the normalization of the smooth-minimum. Note that to the right of the graphs I've noted the value of the kernel at the origin, that is, the height of its intersection with the y axis. This is because that's the value we are subtracting from  $b$  when we are equidistant to the two SDFs  $a$  and  $b$ , that is, when  $x = (b-a)/k = 0$ , because  $a=b$ . That's the value we need to normalize all the smooth-minimums of this family against, for it is the thickest part of the blend region created between the source SDFs.

If you'd like to compute these normalization values yourself, please note that in the case of the Sigmoid kernel you'll get a zero-divided-by-zero value, so you'll need to use L'Hopital's rule to get the correct answer.

Now, some of the Kernels  $g(x)$  above never take the exact reach 0.0 nor  $x$  exactly, no matter how large  $x$  is in absolute value. These are the **exponential**, **sigmoid** and **square root** smooth-minimums, and that's the reason these three never return exactly neither  $a$  nor  $b$ , no matter how far and close these shapes are to each other.

That in turn means that the shapes of the SDFs **a** and **b** will be distorted everywhere in space, not just in the neighborhood of the region of closest contact. I call this phenomenon "lack of Rigidity". On the other hand, the Quadratic and Circular smooth-minimums use kernels to which we added extra constraints that prevents this problem, and produce Rigid smooth-minimums, putting them in a different sub-family of smooth-minimums: the "CD Family":

## The CD Family

---

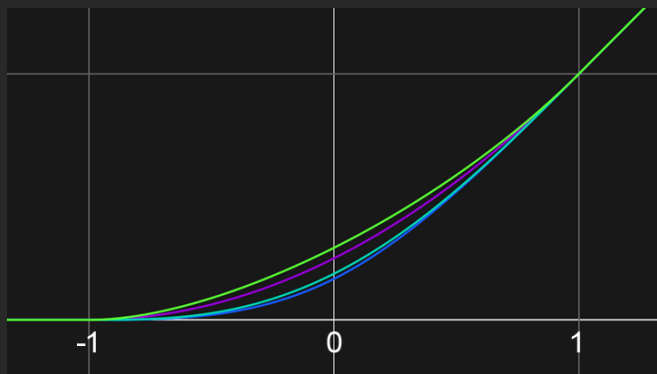
This is the subset of smooth-minimums from the DD Family that force the kernel to connect perfectly with the identity and zero curves. Mathematically, their kernels satisfy these equations:

$$g(-1) = g'(-1) = 0$$

$$g(1) = g'(1) = 1$$

These constraints effectively clamp the effect of the difference of distances, so that the smooth-minimum doesn't have any effect on the SDF shapes **a** nor **b** once they are sufficiently far from each other, ie, **k** units apart or further. This is why I'm calling this the "Clamped Differences Family" or "CD Family".

Naturally you can create an infinite variety of such kernels with these constraints, and therefore there are an infinite amount of possible smooth-minimums that you can use, but this article and the graph below show only a few of the ones I think are most relevant (quadratic, cubic, quartic, circular):



A few CD kernels

Kernel:

$$g(x) = \frac{(x \cdot (2+x) + 1)}{4}$$

$$g(x) = \frac{(1 + 3x \cdot (x+1) - |x^3|)}{6}$$

$$g(x) = \frac{(x+1)^2 (3 - x \cdot (x-2))}{16}$$

Normalization factor:

$$g(0) = 1/4$$

$$g(0) = 1/6$$

$$g(0) = 3/16$$

$$g(0) = 1 - \sqrt{1/2}$$

$$g(x) = 1 + \frac{1}{2}(x - \sqrt{(2 - x^2)})$$

For the sake of clarity, let me list again some of the smooth-minimums in the the CD family:

// quadratic polynomial

```
float smin( float a, float b, float k
{
    k *= 4.0;
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - h*h*k*(1.0/4.0);
}
```

// cubic polynomial

```
float smin( float a, float b, float k
{
    k *= 6.0;
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - h*h*h*k*(1.0/6.0);
}
```

// quartic polynomial

```
float smin( float a, float b, float k
{
    k *= 16.0/3.0;
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - h*h*h*(4.0-h)*k;
}
```

// circular

```
float smin( float a, float b, float k
{
    k *= 1.0/(1.0-sqrt(0.5));
    float h = max( k-abs(a-b), 0.0 )/k;
    return min(a,b) - k*0.5*(1.0+h-sqr
```

And quickly as a historical note, I invented (deduced, I'd say) the quadratic-polynomial smooth-minimum somewhere between 2011 and 2012, but I only publish and shared the code in 2013 (in [a few shaders](#) in Shadertoy). Around 2015 the Media Molecule folks making the Dreams game optimized it (or maybe re-developed it, should ask Dave Smith). I've been using their version ever since they published the optimization. I'll however list the original smooth-minimum code here below for refence, since I know it's still circulating around in certain open source circles:

// quadratic polynomial

```
float smin( float a, float b, float k )
{
    float h = clamp( 0.5+0.5*(b-a)/k, 0.0, 1.0 );
    return mix( b, a, h ) - k*h*(1.0-h);
}
```

Now, looking at all the options within the CD family, you might be asking yourself why would I ever consider using the circular variant above given it uses a square root, which can be a bit too slow to evaluate in some situations. We'll analyze and talk about this one later, but for now let's say that this kernel is the only one that leads to a CD smooth-minimum that produces an exactly circular connection between perpendicular objects. In that regard it is equivalent to the circular-geometrical smooth-minimum presented in the first listing in this

article and which is used by some applications; but unlike the circular-geometrical smooth-minimum, the kernel in the CD family does not produce rendering and collision artifacts (more on this later).

## Gradients

---

It is important that we stop here a bit to study the gradients of our smooth-minimum functions. For one side, computing analytic gradients rather than **numerically** can be a much faster way to do lighting or object placement and alignment. But more importantly for this article, it helps us analyze the behavior of our smooth-minimums mathematically. This is because a necessary (but not sufficient) condition for a field to be a Distance Field (such as an SDF) is that the length of its gradient is 1.0 exactly, everywhere. This can sometimes not be achieved, but knowing when we fail at fulfilling this condition, and by how much we fail to do so, is important information that helps us design our rendering algorithms accordingly.

So, lets start the analysis by calling our smooth-minimum  $f(a, b)$ . Then its gradient will depend on the gradients of the input SDFs **a** and **b** like this:

$$\nabla f(a, b) = \frac{\partial f}{\partial a} \nabla a + \frac{\partial f}{\partial b} \nabla b$$

To check whether the resulting smooth-minimum **f** is also an SDF, we can look at its gradient, see if it has a length of 1. We can check that by computing its squared length:

$$\|\nabla f\|^2 = \left( \frac{\partial f}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial f}{\partial b} \frac{\partial b}{\partial x} \right)^2 + \left( \frac{\partial f}{\partial a} \frac{\partial a}{\partial y} + \frac{\partial f}{\partial b} \frac{\partial b}{\partial y} \right)^2$$

which expands into a quite large expression. But luckily we can group terms and get:

$$\|\nabla f\|^2 = \left( \frac{\partial f}{\partial a} \right)^2 \|\nabla a\|^2 + \left( \frac{\partial f}{\partial b} \right)^2 \|\nabla b\|^2 + 2 \frac{\partial f}{\partial a} \frac{\partial f}{\partial b} (\nabla a \cdot \nabla b)$$

If the input SDFs **a** and **b** are both truly SDFs and not just approximated or bounded SDFs, then

$$\|\nabla a\|^2 = \|\nabla b\|^2 = 1$$

which simplifies the squared length of the smooth-minimum gradient to

$$\|\nabla f\|^2 = \left(\frac{\partial f}{\partial a}\right)^2 + \left(\frac{\partial f}{\partial b}\right)^2 + 2\frac{\partial f}{\partial a}\frac{\partial f}{\partial b}(\nabla a \cdot \nabla b)$$

Looking at this formula one thing is obvious - the length of the gradient depends on the relative orientation of the two SDF's gradients  $\nabla a$  and  $\nabla b$ . Now, armed with this information, let's analyze how the different smooth-minimum functions behave:

## Gradients of the DD and CD family

---

In the case of the DD family we can move a bit more further, since we know that

$$f(a, b) = b - k_n g(x) \quad \text{with} \quad x = \frac{b - a}{k_n}$$

Therefore we can compute the partial derivatives in terms of the Kernel  $g(x)$  like this:

$$\frac{\partial f}{\partial a} = k_n \frac{\partial g((b - a)/k_n)}{\partial a} = k_n \frac{g'}{k_n} = g'$$

$$\frac{\partial f}{\partial b} = 1 - k_n \frac{\partial g((b - a)/k_n)}{\partial b} = 1 - \frac{g'}{k_n} = 1 - g'$$

This leads to the very compact expression for the gradient of the smooth-minimum:

$$\nabla f = g' \nabla a + (1 - g') \nabla b$$

$$\|\nabla f\|^2 = 1 - 2g'(1 - g')(\nabla a \cdot \nabla b - 1)$$

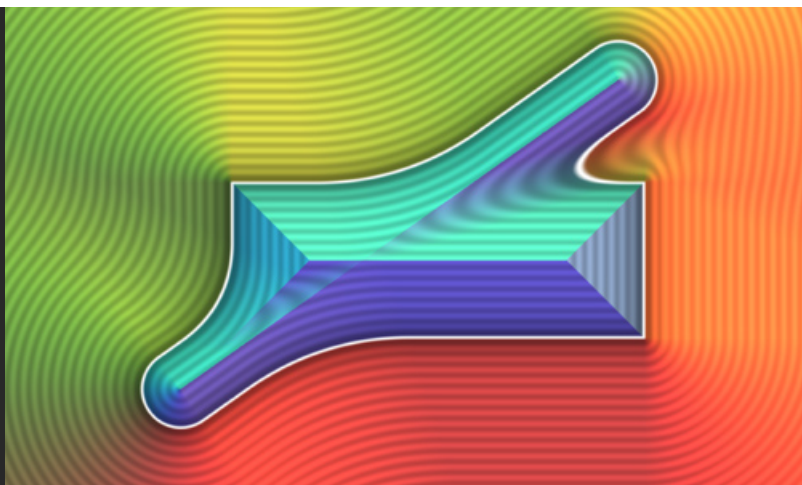
Remember that for our Kernels, we always have  $0 \leq g' \leq 1$ . So according to the first equation above, the gradient of the smooth-minimum is the linear interpolation of the gradients of the input SDFs **a** and **b**. We know that other than at the extremes, the linear interpolation of two vectors is shorter than the two vectors (that's the reason we use `slerp` to interpolate quaternions/rotations, rather than `lerp`). This means that while the smooth-minimums in the DD family will never produce a correct SDF in the blended region, because the length is shorter than 1 we know that they will never overestimate the true distance, and therefore we can use them safely for raymarching and collision detection!

We can also conclude the same thing by noting that in the second equation the parabola  $2g'(1-g')$  is positive in the interval  $0 \leq g' \leq 1$  and that the dot product  $\nabla a \cdot \nabla b \leq 1$ , making the whole expression smaller than 1.

For the CD family of smooth-minimums, remember that outside the blending region where  $|a-b| > k$ , the Kernel **g**(x) is exactly 0 or x, which has derivatives 0 and 1 respectively, making the length of the gradient  $|\nabla f|=1$ . This means that the DD smooth-minimums are exact outside the blending regions, which we already knew since we constructed them explicitly to behave that way.

Since the Quadratic smooth-minimum is so common, it might be worth writing explicitly its gradient. Now, since the quadratic polynomial's Kernel is  $g(x)=(x \cdot (2+x)+1)/4$ , we have that  $g'(x)=(x+1)/2$ . This results in the following code which can be used to compute both the smooth-minimum and its gradient in a single go, assuming now that the variables **a** and **b** bundle both the value of the SDF in **.x** and its gradient in **.yzw** (note again the code below already accounts for the normalization factor **g(0)**):

```
// .x = f(p)
// .y = ∂s(p)/∂x
// .z = ∂s(p)/∂y
// .w = ∂s(p)/∂z
// .yzw = ∇s(p) with |∇s(p)|<1 sadly
vec4 smin( in vec4 a, in vec4 b, in float k )
{
    k *= 4.0;
    float h = max(k-abs(a.x-b.x),0.0)/(2.0*k);
    return vec3(min(a.x,b.x)-h*h*k,
                mix(a.yz,b.yz,(a.x<b.x)?h:1.0-h));
}
```



The gradient of `smin()` in 2D

[Code in Shadertoy](#)

Having analytic gradients is more common than you might think for applications where you want to compute surface normals during the raymarch itself (for **lighting volumetric** shapes or for positioning elements in the scene), or for Machine Learning, without having to implement a full auto-differentiation framework. For examples you can visit the **Distance+Gradient 2D SDF** article where I've compiled a list of primitive distances and their analytic gradients computed efficiently by reusing expressions between both.

## Circular Profile

---

Alright, earlier in the article I briefly mentioned that the particular shape of the blended region can be an important thing to consider. Different smooth-minimums will produce different profiles, even after normalization. And when doing mechanical modeling in particular, it can be useful to ensure that the profile of the blended region is perfectly circular. The Circular Geometric smooth-minimum achieves this objective explicitly, by construction. Unfortunately, it cannot be used for raymarching and collision detection as-is, for it overestimates distances.

So this is where the Circular variant of the CD family comes to the rescue. It achieves mathematically perfect circular profiles as well, but does not suffer from the overestimation problem since it belongs in the CD family, and hence is suitable for rendering. It is however a non-local blend, like all CD and DD smooth-minimums. But let's see how I deduced the formula for the Circular smooth-minimum:

First, we assume we are working with two SDFs **a** and **b** meet at a 90 degree angle, and that **k=1**, so that:

$$x = b - a$$

Now, we want to match the profile of a perfect circle passing through (1,1) and with radius of 1:

$$(a - 1)^2 + (b - 1)^2 = 1$$

But because **b = a + x**, we can substitute in the equation above and get a quadratic polynomial in **a**:

$$2a^2 + a(2x - 4) + (1 - 2x + x^2) = 0$$

which we can solve with the quadratic equation formula to get

$$a = 1 - \frac{x - \sqrt{2 - x^2}}{2}$$

Great. Now we need to match the gradient of **f** to that of the circle. From our analysis on gradients we did earlier, we know that our CD Circular smooth-minimum will have a gradient of the form

$$\nabla f = g' \nabla a + (1 - g') \nabla b$$

But since our SDFs **a** and **b** are vertical and horizontal planes, we have  $\nabla \mathbf{a} = (1, 0)^T$  and  $\nabla \mathbf{b} = (0, 1)^T$ , so  $\nabla \mathbf{f} = (g'(x), 1 - g'(x))^T$ . Also, the (unnormalized) gradient of our circle  $\nabla f$  points in the direction  $(1-a, 1-b)^T$  since it's a circle centered at (1,1). So, we can write

$$\begin{cases} g'(x) = m(1 - a) \\ 1 - g'(x) = m(1 - b) \end{cases}$$

where **m** is some constant of proportionality. We can isolate **m** from the first equation

$$m = \frac{g'(x)}{1 - a}$$

and replace it in the second to obtain an expression for  $g'(x)$  in terms of  $a$  and  $b$ :

$$g'(x) = \frac{1-a}{2-a-b}$$

But we already computed both  $a$  and  $b$  in terms of  $x$  in the quadratic above, so we can replace them and after a couple of simplifications, we get

$$g'(x) = \frac{1}{2} \left( \frac{x}{\sqrt{2-x^2}} + 1 \right)$$

This is fantastic, all we need to do now is integrate

$$g(x) = \int g'(x) dx + C$$

and find the right constant  $C$ , to get our final result - a CD Kernel that produces a perfectly circular shape when smoothly blending to perpendicular SDFs:

$$g(x) = 1 + \frac{x - \sqrt{2-x^2}}{2}$$

This Kernel can also be written as  $g(x) = 1 + \sin(\arcsin(x\sqrt{1/2}) - \pi/4)$ , but there isn't any reason to use inverse trigonometric functions when a square root will just do. In fact, if you are a fellow oldschool programmer who got accustomed to CPU coding, the use of a square root might still be a red flag for you. However, it should not be - GPUs perform square roots almost in the same number of cycles as multiplications, unlike the CPUs where the gap is at least an order of magnitude. Still, if you are using CPUs after all for SDF evaluation and really really want to squeeze every single clock cycle available, then you can probably approximate the whole Circular kernel  $g(x)$  with a polynomial. For example, if we take a degree 4 polynomial

$$\begin{aligned} g(x) &= a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0 \\ g'(x) &= 4a_4x^3 + a_3x^2 + 2a_2x + a_1 \end{aligned}$$

then we have 5 coefficients/constraints to determine, from  $a_4$  to  $a_0$ . But because our approximation will be of

CD family, we already have four such constraints  $g(-1) = g'(-1) = 0$  and  $g(1) = g'(1) = 1$ . The last constraint can be simply that  $g(0)$  matches that of the circular kernel, that is,  $g(0) = 1 - \sqrt{1/2}$ . With that we are ready to set the system of equations

$$\begin{array}{rclclclclcl} g(0) & = & & & & & a_0 & = & 1 - \sqrt{1/2} \\ g(1) & = & a_4 & + & a_3 & + & a_2 & + & a_1 & + & a_0 & = & 1 \\ g(-1) & = & a_4 & - & a_3 & + & a_2 & - & a_1 & + & a_0 & = & 0 \\ g'(1) & = & 4a_4 & + & 3a_3 & + & 2a_2 & + & a_1 & & & = & 1 \\ g'(-1) & = & -4a_4 & + & 3a_3 & - & 2a_2 & + & a_1 & & & = & 0 \end{array}$$

which doesn't take much work to solve:

$$\begin{array}{lll} a_4 = 3/4 - \sqrt{1/2} & a_3 = 0 & a_2 = -5/4 + 2\sqrt{1/2} \\ a_1 = 1/2 & a_0 = 1 - \sqrt{1/2} & \end{array}$$

This gives us all we need to code the circular approximation smooth-minimum, either in normal form (left) or in optimized form (right):

```
// circular approximation
float smin( float a, float b, float k
{
    k *= 1.0/(1.0-sqrt(0.5));
    const float a4 = 3.0/4.0 - sq
    const float a2 = -5.0/4.0 + 2.0*sq
    const float a0 = 1.0 - sq
    float x = (b-a)/k;
    float g = (x> 1.0) ? x :
               (x<-1.0) ? 0.0 :
               x*(x*(x*x*a4+a2)+0.5)+a0
    return b - k * g;
}
```

```
// circular approximation
float smin( float a, float b, float k
{
    k *= 1.0/(1.0-sqrt(0.5));
    float h = max( k-abs(a-b), 0.0 )/k
    const float b2 = 13.0/4.0 - 4.0*sq
    const float b3 = 3.0/4.0 - 1.0*sq
    return min(a,b) - k*h*h*(h*b3*(h-4
}
```

To get the code for the optimized form, all you need to do is propagate  $x=h-1$  into the polynomial.

Now, how does this compare to the exact circular smooth-minimum? Let's test it with two rectangles and a large blend band  $k$ . On the left you can see the quadratic smooth-minimum, on the right is the circular (which is exact), and in the middle you see the 4-th degree approximation that we just computed. The red arc is the ideal circular blend section. If you look closely you'll see the approximation doesn't conform to it exactly, there

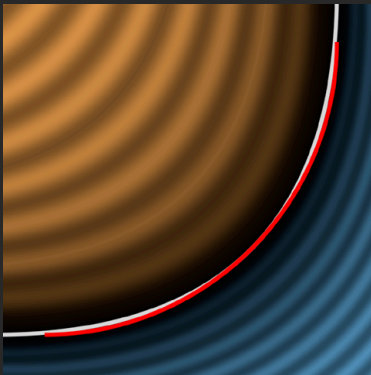
are a few white pixels that leak outside the red arc. In applications where this is a problem you might want to go to a degree 6 polynomial perhaps:



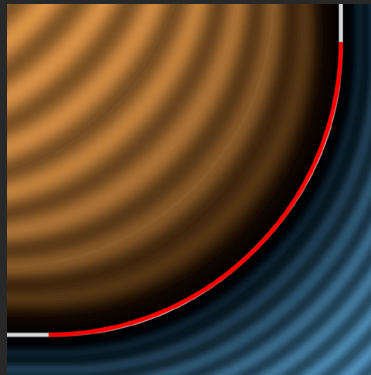
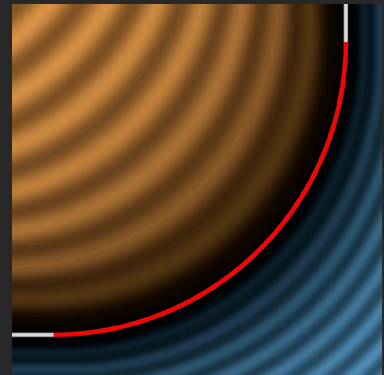
Quadratic smooth-minimum

Circular Approximation  
smooth-minimum

Circular smooth-minimum



Quadratic smooth-minimum

Circular Approximation  
smooth-minimum

Circular smooth-minimum

## Gradients of the Circular Geometrical

The Circular Geometrical smooth-minimum, operates very differently to the DD family. I saw it first in Shadertoy used by folks learning from the demoscene group Mercury. Instead of relying on the difference of distances  $\mathbf{a}$  and  $\mathbf{b}$  like the DD family, it assumes these two SDFs are perpendicular and it explicitly constructs a circle connecting them, centered at  $(\mathbf{k}\mathbf{k}, \mathbf{k})^T$  with radius  $\mathbf{k}$ . So let's do some math see how that works, and without loss of generality, assume that  $\mathbf{k}$  has already been normalized by  $1-\sqrt{1/2}$ :

$$f(a, b) = \begin{cases} \min(a, b) & \text{if } a > k \text{ or } b > k \\ k - \sqrt{(a - k)^2 + (b - k)^2} & \text{otherwise (blending region)} \end{cases}$$

So the partial derivatives of  $f$  with respect to  $a$  and  $b$  are:

$$\frac{\partial f}{\partial a} = \frac{k - a}{\sqrt{(a - k)^2 + (b - k)^2}} \quad \frac{\partial f}{\partial b} = \frac{k - b}{\sqrt{(a - k)^2 + (b - k)^2}}$$

which means that the gradient of the Circular Geometrical smooth-minimum is:

$$\nabla f = \frac{\partial f}{\partial a} \nabla a + \frac{\partial f}{\partial b} \nabla b = \frac{(k - a) \nabla a + (k - b) \nabla b}{\sqrt{(a - k)^2 + (b - k)^2}}$$

and its length squared is

$$\|\nabla f\|^2 = 1 + \frac{2(k - a)(k - b)}{(k - a)^2 + (k - b)^2} (\nabla a \cdot \nabla b)$$

The only way for this quantity to be less than one (which is required to prevent overestimating distances) is:

$$\frac{2(k - a)(k - b)}{(k - a)^2 + (k - b)^2} (\nabla a \cdot \nabla b) \leq 0$$

The denominator is positive and the numerator cannot be negative since inside the blending region both  $a \leq k$  and  $b \leq k$ , making their product positive. So the only regions of space where the gradient's length is less than one is given by

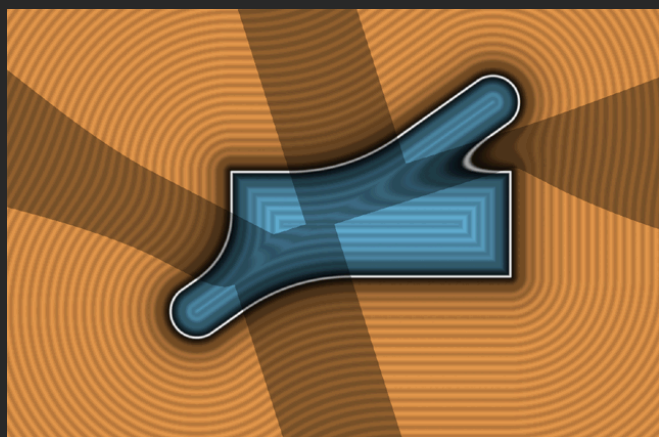
$$\nabla a \cdot \nabla b \leq 0$$

That is the concave regions of space, and only there the Circular Geometrical smooth-minimum underestimates. In other words, in the convex areas of the the union of two shapes the Circular Geometrical smooth-minimum overestimates the distance to the surface of the blended object, and vanilla raymarchers will fail and produce rendering artifacts, as we are about to see.

## Regions of distance under/over-estimation

---

Here's a diagram showing how the CD smooth-minimums and the Circular Geometrical smooth-minimum behave. I've used the Quadratic Polynomial smooth-minimum as a representative of the CD family, but they all have very similar behavior diagrams.



Quadratic smooth-min



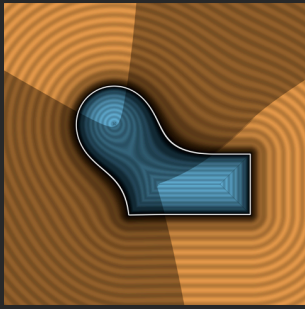
Circular Geometrical smooth-min

Here the shadowed regions of the plane indicate areas where the smooth-minimum returns a lower bound of the real distance to the resulting shape (white boundary). Again, these are regions where the length of the gradient is less than 1.0, where sampling the smooth-minimum (indicated by the yellow dots) produces distance bounds (yellow circles) that do not touch the shape. This means in these regions a raymarcher will perform slower than in the non-shadowed areas where it can traverse the scene at the Speed of Light.

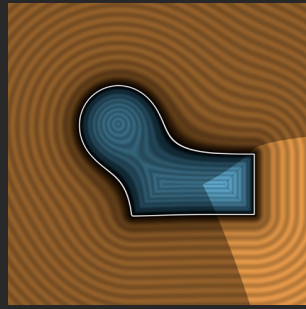
To make things worse, the regions of under-estimation for ALL smooth-minimum belonging on the DD and CD families span to infinity, ie, non-local. Indeed, these shaded regions in the left image only get larger and larger as we move away from the shapes, which is easy to prove: if you split the plane in a Voronoi diagram where each point in the plane is associated with the shape **a** or **b** that is closest to it, you will always find a band where  $|a-b| < k$  that extends along the edges of this Voronoi graph.

This is bad news for our CD Family, which again, depending on the use of the SDFs, can make certain rendering or collision detection methods that rely on bounding boxes be slightly more difficult than they could be otherwise. But not all CD smooth-minimums are equally bad, here you can see a comparison of the areas

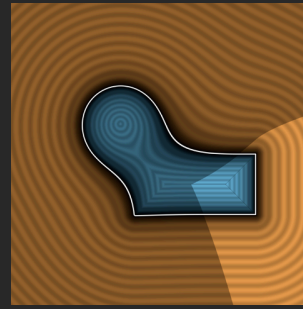
of sub-Speed-of-Light for different variants (the less dark shadowed yellow, the better):



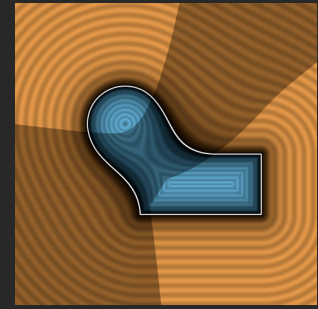
Quadratic



Cubic



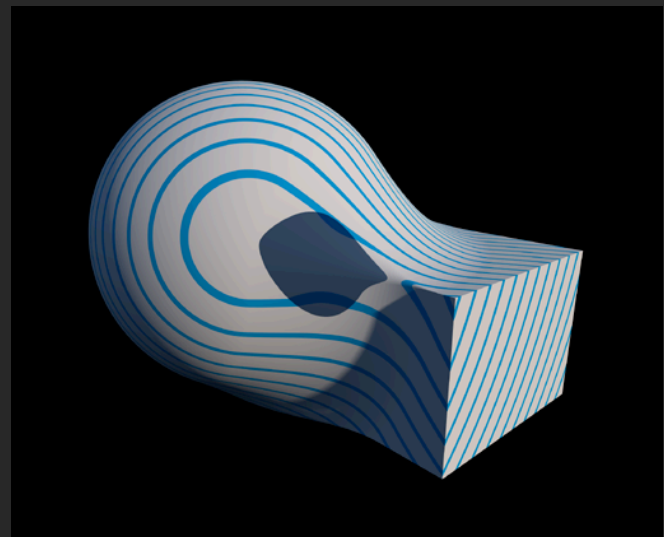
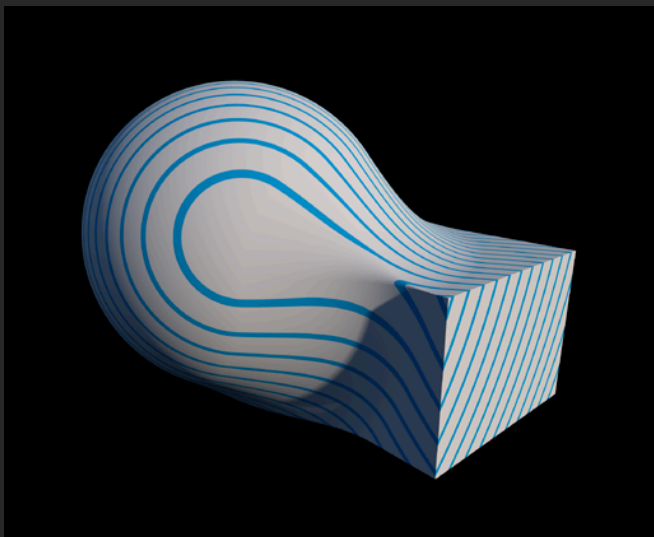
Quartic



Circular

On the other hand, the Circular Geometrical smooth-minimum is locally supported unlike the CD family, and has its underestimation and overestimation regions bounded, which you can see by seeing that when far enough from the smooth-minimum shape, the plane is all regular yellow and isolines are exact. Keeping the slow raymarching regions contained to around the objects is a great property of course.

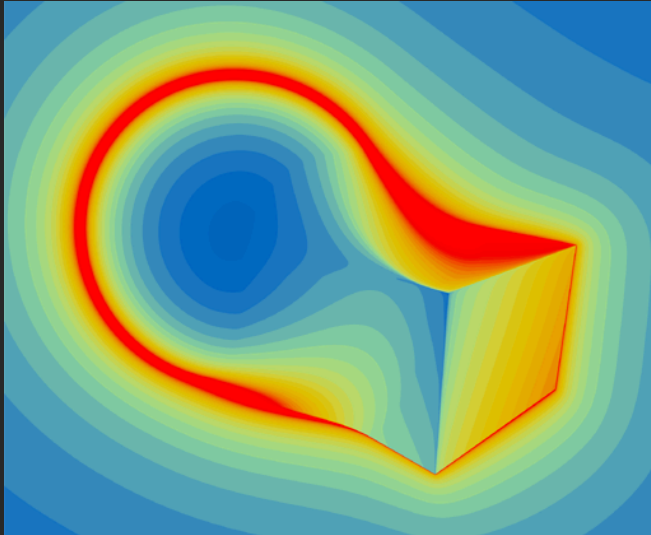
However, here's the big drawback of the Circular Geometrical smooth-minimum - as we mentioned earlier, it often overestimates distances. The red colors indicate regions where the smooth-minimum overestimates the distance to the shape, which is very dangerous. I've marked in light red those where the length of the gradient is larger than one as discussed earlier, and in darker red regions where the gradient has length one **yet** the smooth-minimum is returning the distance to the underlying SDFs **a** or **b** rather than to the blended shape. Sampling the smooth-minimum in those regions produce distance bounds (yellow circles) that penetrate into the shape, or in other words, these are regions where raymarcher will go faster than the Speed of Light and things will break badly and renders will fail by showing holes on the objects, like seen in the image on the right in the following comparison:



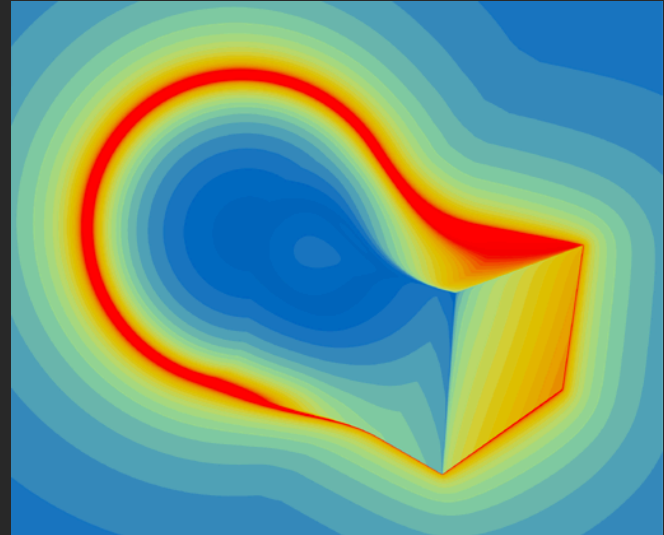
## Circular smooth-min

## Circular Geometrical smooth-min

While this seems bad, it can sometimes be easy to remedy by letting the raymarcher back-track when penetrating the surface, which can be done by breaking the loop based on the absolute value of the distance to the surface rather than the signed distance. The effort might be worth it, considering the Locality of the Circular Geometrical smooth-minimum which translates into less raymarching iterations to render the same picture. Here in the image below blue regions indicate the intersection was found in less than 15 iterations, yellow less than 25, and red 30 or more:



Circular smooth-min



Circular Geometrical smooth-min

You can see that the CD Circular smooth-minimum required more iterations to resolve than the Circular Geometrical, specially around the center of the image.

## Properties

---

Okey, let's gather all the things we've talked about so far about smooth-minimum, and make a list of the properties we care most about, one by one.

**Rigidity** : Some smooth-minimum will distort the SDFs **a** and **b** no matter how far they are from each other. However, the CD family of smooth-minimums and the Circular Geometrical do preserve the shape of the SDFs **a** and **b** everywhere but in the blending region.

**Locality** : all smooth-minimum will always produce non-exact SDF in some regions. As we saw, for the CD Family there are always infinitely large regions where the smooth-minimum under-estimates the distance, no matter how far we move away from the shapes or how far apart these shapes are. On the other hand, the Circular Geometric smooth-minimum **has** Local Support and doesn't suffer from this problem, making it an attractive option depending on the application.

**Conservative** : smooth-minimums that never overestimate distances are Conservative. This is a very desirable property, for it makes algorithms like raymarching and collision detection easier to implement in a robust manner without producing visual artifacts. Fortunately, ALL the smooth-minimums in the CD Family are guaranteed to produce underestimates by constriction, both because the kernel  $g(x) \leq 0$  and also because the length of their gradient is always less than one. However, the Circular Geometrical smooth-minimum violates this property in many regions of space as we saw earlier, making it a less attractive option in some applications.

**Associativity** : the CD Family of smooth-minimums is not associative, in that  $\text{smin}(\mathbf{a}, \text{smin}(\mathbf{b}, \mathbf{c})) \neq \text{smin}(\text{smin}(\mathbf{a}, \mathbf{b}), \mathbf{c})$ . That is, the order in which you blend objects matters. This is certainly true for all members of the DD family. However, the Exponential and the Circular Geometrical can be blended in any order!

So here goes a summary:

|                    | Rigid | Local | Cons. | Asso. |
|--------------------|-------|-------|-------|-------|
| Quadratic          | Yes   | No    | Yes   | No    |
| Cubic              | Yes   | No    | Yes   | No    |
| Quartic            | Yes   | No    | Yes   | No    |
| Circular           | Yes   | No    | Yes   | No    |
| Exponential        | No    | No    | Yes   | Yes   |
| Sigmoid            | No    | No    | Yes   | No    |
| Root               | No    | No    | Yes   | No    |
| Circular Geometric | Yes   | Yes   | No    | Yes   |

As you can see, there's no single smooth-minimum I know of that has all the desired properties, so the choice will be depend on your and what are the requirements of the application using it.

## Results

---

In general I use the Quadratic polynomial smooth-min function because it's fast, close enough to circular, never overestimates, and while not local it still not too bad. Although the Circular is also a great option. Regardless, here go a few examples of the many uses I've done of the Quadratic smooth-minimum in the past to connect surfaces, such as snow and bridge in the images below.



Regular min()



Polynomial smooth-min()

Note how the snow gently piles by the bridge thanks to the smooth minimum, and how thanks to the mix factor described above we can transition between the snow and stone materials (source code and realtime demo: [shadertoy.com/view/Mds3z2](https://shadertoy.com/view/Mds3z2)).



Regular min()



Polynomial smooth-min().

Note how the regular min() union cannot smoothly connect the legs of the creature to its body (source code and realtime demo: [shadertoy.com/view/Mss3zM](https://shadertoy.com/view/Mss3zM)). Naturally, the technique is very handy for connecting the different pieces of one same character, such as the arms, head and body, which in the case of the following realtime shader are made of spheres, ellipsoids and segment primitives):

inigo quilez - learning computer graphics since 1994